

Manual de Instalação

Projeto ArenaPlay (trabalhoprogweb) · Do download até o sistema rodando
Windows (WAMP ou XAMPP) e Linux

1. O que este manual cobre

Este manual leva você do **download do projeto** até o **sistema funcionando** na máquina, na ordem correta, dizendo exatamente o que preencher.

O que este manual NÃO faz: ele não ensina a instalar o WAMP, o XAMPP, o Node.js ou o Git — apenas informa **o que precisa existir na máquina** e como conferir. A instalação desses programas segue a documentação oficial de cada um.

2. Requisitos da máquina

Confira cada item **antes** de começar. Um requisito faltando causa erros confusos lá na frente.

Componente	Versão	Por que é necessário	Como conferir
PHP	8.3 ou superior	Exigido pelo <code>composer.json</code> (<code>"php": "^8.3"</code>). Versões menores nem instalam as dependências.	<code>php -v</code>
Composer	2.x	Instala as bibliotecas PHP (Laravel, Jetstream, Sanctum).	<code>composer --version</code>
MySQL ou MariaDB	MySQL 8.0+ (mínimo 5.7) MariaDB 10.2+	Uma das migrations cria uma coluna gerada (STORED) , recurso que não existe em versões antigas. Nelas o <code>migrate</code> falha.	<code>mysql --version</code>
Node.js + npm	Node 18+	Compila os arquivos de estilo de 7 telas (recuperar senha, verificar e-mail, termos, política...). Sem isso, essas páginas quebram.	<code>node -v</code> e <code>npm -v</code>
Git	qualquer	Baixar o projeto.	<code>git --version</code>

2.1 Extensões do PHP

Todas costumam vir habilitadas no WAMP e no XAMPP, **menos a GD em algumas instalações Linux**.

Extensão	Para que serve neste projeto
<code>gd</code>	Obrigatória. Redimensiona e comprime as fotos do cabeçalho da tela inicial. Sem ela, o envio de foto falha.
<code>pdo_mysql</code>	Conexão com o banco de dados.
<code>mbstring</code>	Textos com acento (todo o sistema é em português).
<code>openssl</code>	Criptografia de senhas e sessões.
<code>fileinfo</code>	Validação real do tipo de arquivo enviado (segurança do upload).
<code>tokenizer</code> , <code>xml</code> , <code>ctype</code> , <code>json</code> , <code>curl</code> , <code>bcmath</code> , <code>zip</code>	Exigências padrão do Laravel.

Para ver **todas as extensões ativas**, rode:

```
php -m
```

Para conferir se **uma** específica está ativa (troque `gd` pelo nome da extensão):

Terminal	Comando
Windows (PowerShell)	<code>php -m Select-String gd</code>
Windows (Prompt / cmd)	<code>php -m findstr gd</code>
Linux	<code>php -m grep gd</code>

Se o nome aparecer na saída, a extensão está ativa. Se não aparecer, siga a caixa abaixo.

Como habilitar uma extensão: abra o `php.ini` , encontre a linha `;extension=gd` e **remova o ponto-e-vírgula** do início. Depois reinicie o servidor. Para descobrir qual `php.ini` está em uso: `php --ini` .

3. Windows (WAMP ou XAMPP)

3.1 Onde colocar o projeto

Programa	Pasta do projeto	Endereço no navegador
WAMP	C:\wamp64\www\trabalhoprogweb-main	http://localhost/trabalhoprogweb-main/public
XAMPP	C:\xampp\htdocs\trabalhoprogweb-main	http://localhost/trabalhoprogweb-main/public

Modo mais simples: você **não precisa** do Apache. Rodando `php artisan serve` dentro da pasta do projeto, o sistema abre em `http://127.0.0.1:8000`. O WAMP/XAMPP fica servindo apenas o **MySQL**. Neste modo, a pasta do projeto pode ficar em qualquer lugar.

3.2 Atenção: existem DOIS php.ini

Este detalhe causa muita confusão:

- O **Apache** usa um `php.ini` (ex.: `C:\wamp64\bin\apache\...\bin\php.ini`).
- A **linha de comando** (`php artisan serve`) usa outro (ex.: `C:\wamp64\bin\php\php8.3.x\php.ini`).

Se você habilitar uma extensão no arquivo errado, nada muda. Rode `php --ini` para ver qual arquivo o terminal está lendo — é o que importa quando se usa `php artisan serve`.

3.3 Colocando o MySQL para rodar

Inicie o WAMP/XAMPP e verifique se o serviço do **MySQL** está ativo (ícone verde no WAMP; botão *Start* ao lado de MySQL no painel do XAMPP). O usuário padrão costuma ser **root sem senha** — é o que o `.env.example` já assume.

4. Linux

Os pacotes variam por distribuição. O que precisa existir:

O que instalar	Observação
PHP 8.3 e as extensões da seção 2.1	Em Debian/Ubuntu, cada extensão é um pacote separado: <code>php8.3-mysql</code> , <code>php8.3-gd</code> , <code>php8.3-mbstring</code> , <code>php8.3-xml</code> , <code>php8.3-curl</code> , <code>php8.3-zip</code> , <code>php8.3-bcmath</code> .
MySQL Server ou MariaDB	Respeite a versão mínima da seção 2.
Composer	—
Node.js e npm	—
Git	—

Permissões — o erro nº 1 no Linux. O servidor web precisa **escrever** em duas pastas. Sem isso o sistema mostra tela branca ou erro 500 logo de cara:

```
sudo chown -R $USER:www-data storage bootstrap/cache  
sudo chmod -R 775 storage bootstrap/cache
```

No Windows isso não é necessário.

5. Instalação passo a passo

Execute **nesta ordem**. Cada passo depende do anterior.

1 Baixar o projeto

```
git clone https://github.com/renatorodrigues3333/trabalhoprogweb.git
cd trabalhoprogweb
```

2 Instalar as dependências do PHP

```
composer install
```

Cria a pasta `vendor/`. Demora alguns minutos na primeira vez.

Não rode `composer update` antes. É um erro comum e pode quebrar o projeto.

O arquivo `composer.lock` vem no repositório e guarda a **versão exata** de cada biblioteca que foi testada com este código. O `composer install` lê esse arquivo e instala **exatamente** essas versões — é o que garante que o projeto funcione igual na sua máquina.

Já o `composer update` **ignora** o que está travado, busca as versões mais novas disponíveis e **reescreve o `composer.lock`**. Uma biblioteca atualizada pode ter mudado de comportamento e o sistema passa a apresentar erros que não têm nada a ver com a sua instalação.

Regra: `install` para *usar* o projeto · `update` só quando você quiser *deliberadamente* atualizar uma biblioteca — e aí o `composer.lock` novo deve ser testado e commitado.

Se o `composer install` falhar dizendo "Your lock file does not contain a compatible set of packages. Please run `composer update`", não obedeça a essa sugestão. Veja a seção 8 — obedecê-la já quebrou o projeto uma vez.

3 Criar o arquivo de configuração

```
Windows: copy .env.example .env
Linux:   cp .env.example .env
```

O `.env` guarda senhas e por isso **não vem no repositório** — ele precisa ser criado em cada máquina.

4 Gerar a chave da aplicação

```
php artisan key:generate
```

Não pule este passo. Sem o `APP_KEY`, o Laravel não consegue criptografar as sessões e você vai receber "419 Page Expired" ao tentar entrar em qualquer formulário.

5 Preencher o `.env`

Abra o `.env` num editor de texto e confira os campos da **seção 6** deste manual. Se o seu MySQL tem senha no `root`, é aqui que você a informa.

6 Criar o banco e as tabelas

```
php artisan migrate
```

Você NÃO precisa criar o banco à mão. Se o banco `arena_play` não existir, o Laravel avisa e pergunta: *"The database 'arena_play' does not exist. Would you like to create it?"*. Responda **yes** e ele cria o banco e todas as tabelas.

Em servidor (sem alguém para responder), use `php artisan migrate --force`, que cria sem perguntar.

7 Popular os dados iniciais

```
php artisan db:seed
```

Este passo é **obrigatório**. Ele cria:

- As **formas de pagamento** (PIX, Cartão, Dinheiro). Sem elas, não é possível concluir o cadastro de uma arena — o formulário exige escolher ao menos uma.
- O **administrador do sistema** (credenciais na seção 9).

O comando é seguro de rodar mais de uma vez: ele não duplica registros.

8 Liberar o acesso às imagens

```
php artisan storage:link
```

O passo mais esquecido. Ele cria um atalho de `public/storage` para `storage/app/public`. Sem ele, as fotos enviadas pelo admin são **salvas**, mas resultam em **erro 404** no navegador. A tela `/admin/aparencia` mostra um aviso vermelho quando detecta que este comando não foi executado.

9 Compilar os arquivos de estilo

```
npm install  
npm run build
```

Também obrigatório. A pasta `public/build` não vem no repositório. Sem estes comandos, **7 telas quebram** com o erro *"Vite manifest not found"*: recuperar senha, redefinir senha, verificar e-mail, confirmar senha, autenticação em duas etapas, termos de uso e política de privacidade.

10 Rodar o sistema

```
php artisan serve
```

Abra `http://127.0.0.1:8000` no navegador. Para parar, pressione `Ctrl + C`.

Se preferir usar o Apache do WAMP/XAMPP, veja os endereços da seção 3.1 — nesse caso não é preciso rodar `php artisan serve`.

6. O que preencher no `.env`

6.1 Campos obrigatórios

Campo	Valor	O que faz
<code>APP_KEY</code>	gerado pelo comando	Chave de criptografia. Vem vazio; o passo 4 preenche.
<code>APP_URL</code>	<code>http://127.0.0.1:8000</code>	Endereço do sistema. Usado nos links dos e-mails . Ajuste se usar o Apache.
<code>DB_CONNECTION</code>	<code>mysql</code>	Tipo de banco.
<code>DB_HOST</code>	<code>127.0.0.1</code>	Onde o MySQL está.
<code>DB_PORT</code>	<code>3306</code>	Porta padrão do MySQL.
<code>DB_DATABASE</code>	<code>arena_play</code>	Nome do banco. Criado sozinho no passo 6.
<code>DB_USERNAME</code>	<code>root</code>	Usuário do MySQL.
<code>DB_PASSWORD</code>	(vazio)	Senha do MySQL. No WAMP/XAMPP o padrão é sem senha .

6.2 Campos que já vêm certos (não mexa sem motivo)

Campo	Valor	Por que está assim
<code>APP_NAME</code>	<code>ArenaPlay</code>	Aparece como remetente dos e-mails.
<code>APP_ENV</code>	<code>local</code>	Modo de desenvolvimento.
<code>APP_DEBUG</code>	<code>true</code>	Mostra os erros na tela. Em produção, mude para <code>false</code> — senão o visitante vê detalhes internos do sistema.
<code>QUEUE_CONNECTION</code>	<code>sync</code>	Envia os e-mails na hora, sem precisar de processo em segundo plano.
<code>MAIL_MAILER</code>	<code>log</code>	Os e-mails não são enviados : são gravados em <code>storage/logs/laravel.log</code> . Ideal para desenvolver sem disparar e-mail real a um cliente.
<code>EMAIL_VERIFICATION_ENABLED</code>	<code>false</code>	Não exige confirmar o e-mail no cadastro. Só ligue depois de configurar o envio real (seção 6.3).
<code>SESSION_DRIVER</code>	<code>file</code>	Sessões em arquivo. Não exige tabela no banco.
<code>CACHE_STORE</code>	<code>file</code>	Cache em arquivo. Se trocar para <code>database</code> , as tabelas <code>cache</code> e <code>cache_locks</code> precisam existir — elas são criadas pelas migrations, mas o <code>.env</code> precisa ser ajustado depois de rodá-las.

O `.env.example` já vem com os valores testados. Ele é a configuração que funciona sem passos extras: cache e sessões em arquivo, fila em `sync`. Copie-o e altere apenas o que a seção 6.1 pede.

Dica — o que mexer nisto quando for para produção:

- **Sessão (`SESSION_DRIVER`)**: em arquivo funciona bem. Trocar para `database` permite *ver e encerrar* os dispositivos conectados de cada conta (útil num sistema com caixa/dinheiro). A tabela `sessions` já existe e se limpa sozinha — sessões paradas por mais de 2 horas são apagadas automaticamente.
- **Cache (`CACHE_STORE`)**: mantenha em `file` — num servidor só, é mais rápido que o banco. Cache em `database` só compensa com vários servidores.
- **Fila (`QUEUE_CONNECTION`)**: em produção use `database` e deixe `php artisan queue:work` rodando — assim o e-mail é enviado *em segundo plano* e não trava a tela de quem confirma uma reserva. As três tabelas de fila já existem no banco: `jobs` (tarefas na fila de espera), `failed_jobs` (tarefas que falharam, para inspecionar e reprocessar) e `job_batches` (progresso de lotes de tarefas). Elas ficam **vazias** enquanto a fila é `sync` (tudo roda na hora).
Atenção: com `database` e sem o `queue:work` rodando, nenhum e-mail sai — os jobs ficam parados na tabela `jobs`. Checklist completo na seção 11.

6.3 Opcional: enviar e-mails de verdade

Com `MAIL_MAILER=log`, nada é enviado. Para enviar mesmo (avisos de reserva, "esqueci minha senha"):

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=seu-email@gmail.com
MAIL_PASSWORD=senha-de-app-de-16-digitos
MAIL_SCHEME=tls
MAIL_FROM_ADDRESS="seu-email@gmail.com"
```

Duas armadilhas:

1. O Gmail **não aceita sua senha normal**. É preciso gerar uma "senha de app" (exige verificação em duas etapas ativa na conta).
2. Só ligue `EMAIL_VERIFICATION_ENABLED=true` **depois** que o envio funcionar. Ligar antes tranca todo cadastro novo fora do sistema, esperando um e-mail que nunca chega.

Depois de qualquer alteração no `.env`, rode: `php artisan config:clear`

6.4 Opcional: permitir fotos maiores

O padrão de fábrica do PHP aceita arquivos de até **2 MB**. O sistema já contorna isso (o navegador encolhe a foto antes de enviar), então **não é obrigatório mexer**. Se quiser folga, edite o `php.ini` e **reinicie o servidor**:

```
upload_max_filesize = 8M
post_max_size = 12M
```

`post_max_size` precisa ser **maior** que `upload_max_filesize`.

7. Verificação final

Rode estes comandos na pasta do projeto. Todos devem passar:

Comando	Resposta esperada
<code>php artisan migrate:status</code>	Lista as migrations, todas com Ran .
<code>php artisan route:list</code>	Lista as rotas sem erro (o sistema sobe).
<code>php artisan tinker --execute="echo DB::getDatabaseName();" </code>	<code>arena_play</code>
Existe a pasta <code>public/storage</code> ?	Sim (criada no passo 8).
Existe a pasta <code>public/build</code> ?	Sim (criada no passo 9).

E no navegador:

- `http://127.0.0.1:8000` → tela inicial com o carrossel de fotos.
- `http://127.0.0.1:8000/login` → entre com o administrador (seção 9).
- `http://127.0.0.1:8000/forgot-password` → se abrir sem erro, o `npm run build` deu certo.

8. Problemas comuns

Mensagem / sintoma	Causa	Solução
<i>Vite manifest not found</i>	Os arquivos de estilo não foram compilados.	Rode <code>npm install</code> e <code>npm run build</code> (passo 9).
<i>SQLSTATE[HY000] [1049] Unknown database</i>	O banco não existe.	Rode <code>php artisan migrate</code> e responda yes . Confira também o <code>DB_DATABASE</code> .
<i>SQLSTATE[HY000] [1045] Access denied</i>	Usuário ou senha do MySQL errados.	Ajuste <code>DB_USERNAME</code> e <code>DB_PASSWORD</code> . Depois: <code>php artisan config:clear</code> .
<i>SQLSTATE[HY000] [2002] Connection refused</i>	O MySQL não está rodando.	Inicie o serviço no painel do WAMP/XAMPP.
Erro 419 Page Expired ao enviar formulário	<code>APP_KEY</code> vazio.	<code>php artisan key:generate</code> (passo 4).
Fotos da tela inicial dão 404	Falta o atalho das imagens.	<code>php artisan storage:link</code> (passo 8).
<i>Call to undefined function imagecreatefromjpeg()</i>	Extensão GD desabilitada.	Habilite <code>extension=gd</code> no <code>php.ini</code> e reinicie.
<i>Não foi possível enviar a imagem... passa do limite</i>	Foto maior que o <code>upload_max_filesize</code> .	Use uma foto menor, ou aumente o limite (seção 6.4).
Tela branca ou erro 500 logo após instalar (Linux)	Sem permissão de escrita.	Corrija as permissões de <code>storage</code> e <code>bootstrap/cache</code> (seção 4).
Não consigo cadastrar uma arena: nenhuma forma de pagamento aparece	O <code>db:seed</code> não foi executado.	<code>php artisan db:seed</code> (passo 7).
O e-mail não chega	<code>MAIL_MAILER=log</code> : ele é gravado, não enviado.	Veja <code>storage/logs/laravel.log</code> , ou configure o SMTP (seção 6.3).
Mudei o <code>.env</code> e nada mudou	Configuração em cache.	<code>php artisan config:clear</code>
Erro 500 ao tentar entrar: <i>SQLSTATE[42S02] ... Tabela 'arena_play.cache' não existe</i>	O <code>.env</code> está com <code>CACHE_STORE=database</code> , mas as tabelas de cache não foram criadas. A tela de login usa o cache para limitar tentativas (<code>throttle:login</code>).	Use <code>CACHE_STORE=file</code> (é o padrão do <code>.env.example</code>) — não depende de tabela. Se quiser mesmo usar o banco, rode <code>php artisan migrate</code> antes, que cria as tabelas <code>cache</code> e <code>cache_locks</code> . Depois: <code>php artisan config:clear</code> .
Erros estranhos logo após instalar (classe não encontrada, método inexistente)	Foi executado <code>composer update</code> , que instalou versões diferentes das testadas.	Restaure o arquivo travado e reinstale: <code>git checkout composer.lock</code> <code>composer install</code>
<i>Your lock file does not contain a compatible set of packages. Please run composer update.</i> (seguido de vários "requires php >= 8.4 — your php version (8.3.x) does not satisfy")	O <code>composer.lock</code> foi gerado numa máquina com PHP mais novo que o seu. Ele travou versões de bibliotecas que a sua versão do PHP não suporta.	Não rode <code>composer update</code> — é o que o Composer sugere, mas ele instala versões diferentes das testadas e pode quebrar o projeto. 1. Confira sua versão: <code>php -v</code> (precisa ser 8.3 ou superior). 2. Se estiver certa, avise quem mantém o projeto: o <code>composer.lock</code> precisa ser

		regerado na versão mínima suportada e commitado.
<i>Call to undefined method Laravel\Fortify\Features::passkeys()</i> (aparece em todo comando <code>artisan</code> , inclusive no <code>key:generate</code>)	Foi executado <code>composer update</code> , que rebaixou o <code>laravel/fortify</code> para uma versão sem esse recurso. Como o <code>config/fortify.php</code> é lido por qualquer comando, tudo para.	Atualize o projeto e restaure as versões travadas: <pre>git pull git checkout composer.lock composer install</pre>

Por que esses dois erros aparecem juntos: o primeiro leva ao segundo. O Composer sugere `composer update` , a pessoa obedece, o update resolve as dependências de outro jeito e derruba um pacote de que a configuração do projeto depende. O resultado é que **nenhum comando `artisan` funciona mais** — nem o `key:generate` do passo 4. A saída é sempre voltar ao `composer.lock` do repositório, nunca gerar um novo por conta própria.

9. Acesso inicial

O comando `php artisan db:seed` cria este administrador:

Campo	Valor
E-mail	<code>adminteste@gmail.com</code>
Senha	<code>12345678</code>
Painel	<code>http://127.0.0.1:8000/admin</code>

Nunca leve esta senha para produção. Ela existe apenas para o primeiro acesso em ambiente de desenvolvimento. Ao publicar o sistema, entre no painel e troque a senha em *Perfil* → *Trocar senha*.

Os outros perfis se criam pelo próprio site:

- **Cliente:** `/register`
- **Proprietário de arena:** `/registerArenaOwners` (cadastro em 4 etapas)
- **Funcionário:** criado pelo proprietário, dentro do painel dele

10. Sobre os arquivos e o banco

Item	Onde fica	Vai para o Git?
Imagem padrão da tela inicial	<code>public/img/home-default.jpg</code>	Sim — garante que a home nunca quebre
Fotos enviadas pelo admin	<code>storage/app/public/slides</code>	Não — são por máquina
Configurações e senhas	<code>.env</code>	Não — criado em cada máquina
Bibliotecas PHP	<code>vendor/</code>	Não — vem do <code>composer install</code>
Estilos compilados	<code>public/build</code>	Não — vem do <code>npm run build</code>

Consequência prática: ao instalar numa máquina nova, o banco pode até vir com os registros das fotos, mas **os arquivos das imagens não vêm junto**. O sistema detecta isso e mostra a imagem padrão no lugar, sem quebrar. Basta reenviar as fotos em `/admin/aparencia`.

Backup: guardar apenas o *dump* do banco **não basta**. É preciso salvar também a pasta `storage/app/public`, onde ficam as imagens enviadas.

11. Antes de publicar em produção

Este manual configura um ambiente de **desenvolvimento**. Para colocar no ar, ajuste:

Campo	Trocar para	Motivo
APP_DEBUG	false	Com <code>true</code> , um erro mostra código e configurações ao visitante.
APP_ENV	production	—
APP_URL	domínio real	Define os links dentro dos e-mails.
MAIL_MAILER	smtp	Para os e-mails saírem de fato.
QUEUE_CONNECTION	database	Não trava a tela enquanto o e-mail é enviado. Exige manter <code>php artisan queue:work</code> rodando — sem isso, nenhum e-mail sai.
Agendador	cron	<code>php artisan schedule:run</code> a cada minuto atualiza os status das reservas. Sem ele o sistema funciona, mas os status só mudam quando alguém abre uma tela.

E troque a senha do administrador.